

M16 — WebSocket transport for synchronized control

Phase: 10 — WS **Status:** DONE (2026-05-26) **Blocked by:** M4 **Blocks:** none **Estimated effort:** 3 days ·
Actual: 0.4 day

1. Purpose

Replace HTTP-pollled freeze/resume/scene/end propagation with a WebSocket push channel per room. Subscribers (the M5 dashboard, chat stations, EHR stations, future Nursing Station) receive events in real time instead of waiting for the next 2 s poll.

The polled state path (`/api/room/state`) stays — it's the authoritative source-of-truth read; WS is the **push** notification. Subscribers that miss a push (offline, just reconnecting) catch up on the next poll automatically.

2. Structure

New files:

- `portal/ws_room.py` — Manager class + `handle_room_ws` handler + 4 emitter helpers (`emit_freeze_all`, `emit_resume_all`, `emit_room_end`, `emit_scene`).

Files touched:

- `portal/server.py`:
- Imports `ws_room` and FastAPI's `WebSocket`.
- `@app.websocket("/ws/room/{room_code}")` endpoint.
- Four M4 routes (`freeze_all`, `resume_all`, `end`, and the two

`scene` routes) now also fire the WS emitters after their state mutation. Errors in WS emission are swallowed (broadcast is best-effort; durable state is the source of truth).

No station-side JS changes in M16 — the spec mentioned updating `station_chat.js` and the EHR React app, but those touch the v6 client surfaces and would balloon M16's blast radius. Subscribers can pick up the WS by adding ~15 lines of JS each when the operator confirms the M21 LAN test wants real-time push on those surfaces. The server-side contract is in place; clients can opt-in incrementally.

3. Uses

- M5 dashboard's `control_room.js` could subscribe to `/ws/room/{room_code}` and skip its 2 s poll when push is active. Not yet wired.
- Phase 7 Nursing Station (M27) will subscribe for live telemetry + alarm updates.
- M28 Intercom will use the WS for voice signaling.

4. Functions (exported API surface)

`portal/ws_room.py`

Symbol	Signature	Purpose
manager	module singleton _RoomManager	Tracks subscribers per room_code.
manager.connect(room_code, ws)	async	Accept + register the subscriber.
manager.disconnect(room_code, ws)	async	Unregister (idempotent).
manager.broadcast(room_code, message)	async → int	Send to every subscriber; returns count of successful sends. Closed sockets pruned silently.
manager.subscriber_count(room_code)	int	For telemetry + tests.
emit_freeze_all, emit_resume_all, emit_room_end, emit_scene	async	Convenience wrappers that build the right envelope.
handle_room_ws(ws, room_code)	async	Endpoint handler — accept, hold, clean up on disconnect.

Wire envelope

```
{
  "type":      "freeze_all" | "resume_all" | "end" | "scene",
  "ts":        <float>,
  "room_code": "ABCDEF",
  "encounter_id": "...",      (optional, scene only)
  "payload":   { ... }       (event-specific)
}
```

WebSocket endpoint

/ws/room/{room_code} — public (no operator vault required); the room_code is the access token, matching the v6 chat-station auth pattern. Sockets that send messages have them discarded — this is a one-way push channel for now (future: presence pings, intercom).

5. Limitations

- **No JS subscribers wired yet.** The server pushes; no client

reads. Surfaces (dashboard, chat station, EHR) keep polling and work fine. M16 is the **enabling** layer; clients pick it up incrementally.

- **No replay buffer.** A subscriber that connects mid-room sees

only events from connect-time forward. The /api/room/state poll provides the catch-up read; that's enough for current use cases.

- **No authentication on the WS upgrade.** room_code IS the

token. Anyone with the code can connect to the broadcast channel. For shared bedside / nurse station deployments this is fine; for a future LAN+ scenario it might need tightening (room-scoped JWT). Out of scope for M16.

- **Best-effort broadcast.** Failures don't surface to the route

caller. If WS delivery is critical for a future feature, that feature should poll /api/room/state in addition.

- **Single-process only.** The Manager is in-memory. A

multi-worker uvicorn deployment would not share subscribers. v7 is intentionally single-process (LAN-bound, 1 instructor); a future scale-out would need a Redis pub/sub backend.

6. Test status

Test file	Cases	Status
test_ws_freeze_event_arrives_at_subscribed_stations.py	4 — freeze event delivery; resume event delivery; end event before singleton clears; broadcasts scoped by room_code.	PASS
test_ws_scene_event_appears_in_chart_within_500ms.py	3 — single-encounter scene push < 500 ms; room-broadcast emits one event per target; disconnect cleans up subscriber state.	PASS

7/7 PASS. **Full v7 suite: 111/111 passing** (up from 104 — +7). **Full v6 regression on v7: 222 passed**, same 6 env-flaky pre-existing failures, **0 v7 regressions**.

7. Change list

Date	Author	Change	Files
2026-05-26	claude-code	New portal/ws_room.py (Manager + emitters + handler). /ws/room/{room_code} WebSocket endpoint. Five M4 route handlers (freeze_all, resume_all, end, two scene routes) now fire WS emitters after state mutation. 7 acceptance tests across 2 files.	portal/ws_room.py, portal/server.py, tests/v7/test_ws_*.py

8. Open questions / known issues

- ****Subscriber-side wiring** (M5 dashboard JS, chat station JS, EHR

React app)** is a thin add: ~15 lines per surface to open the WS and react to event types. Defer to whoever runs M21 LAN test — the polling fallback works for now.

- **Authentication on the WS upgrade** is room_code only. If a

future deployment crosses a hostile network the upgrade should carry a room-scoped JWT.

- **Single-process limitation** is acceptable at v7 scale. The

manager's API mirrors portal/devices/ws.py's shape, so a future Redis backend would be a drop-in.